

Universidade Federal de Alfenas

Processamento Digital de Imagens

Resolução Comentada — Segunda Avaliação (16/06/2025)

Questão 1 (2 pontos)

Explique o objetivo de cada etapa do formato JPEG: DCT, Quantizador, RLE e Huffman.

Contexto e Raciocínio

O JPEG é um formato de compressão com perda. Seu pipeline transforma os dados da imagem em representações cada vez mais compactas. Cada etapa tem um papel específico — é importante entender não só o que cada uma faz, mas **por que** ela está ali nessa ordem.

a) DCT — Transformada Discreta do Cosseno

A imagem é dividida em blocos de 8×8 pixels. A DCT transforma cada bloco do domínio espacial (valores de intensidade dos pixels) para o domínio das frequências. O resultado são 64 coeficientes: os de baixa frequência concentram a maior parte da informação visual percebida pelo olho humano (bordas suaves, tonalidades gerais), enquanto os de alta frequência representam variações finas que o olho mal percebe. A DCT, por si só, **não descarta nenhuma informação** — ela apenas reorganiza os dados para facilitar a etapa seguinte.

b) Quantizador

Esta é a **única etapa com perda de informação** no pipeline JPEG. Cada coeficiente DCT é dividido por um valor de uma tabela de quantização (diferente para luminância e croma) e o resultado é arredondado para o inteiro mais próximo. Coeficientes de alta frequência são divididos por valores maiores, sendo frequentemente zerados. Esse arredondamento é irreversível: na descompressão, multiplicamos de volta pelo mesmo valor, mas os dígitos perdidos no arredondamento não se recuperam. A qualidade do JPEG é controlada justamente pelos valores dessa tabela.

c) Codificador RLE — Comprimento de Corrida

Após a quantização, os coeficientes do bloco 8×8 são percorridos em ordem zigue-zague (da frequência mais baixa à mais alta). Como os coeficientes de alta frequência foram zerados pelo quantizador, surgem longas sequências de zeros consecutivos. O RLE codifica essas sequências como pares (quantidade de zeros, valor seguinte), reduzindo drasticamente a quantidade de dados sem perda adicional.

d) Codificador Estatístico — Huffman

O Huffman atribui códigos binários de comprimento variável aos símbolos que saem do RLE. Símbolos mais frequentes recebem códigos mais curtos; símbolos raros recebem códigos mais longos. Isso é feito construindo uma árvore binária a partir das frequências de ocorrência. O Huffman é, assim como o RLE, uma compressão **sem perda**.

Resposta-síntese: DCT reorganiza os dados; Quantizador descarta informação imperceptível (com perda); RLE comprime zeros consecutivos; Huffman comprime estatisticamente — ambos sem perda adicional.

Questão 2 (2 pontos)

Codifique o texto usando Huffman:

AAAABABBCDAAABABBBAAEAABCAAAAABADCCAAABBBAAABBB

Passo 1 — Contagem de frequências

Contamos a ocorrência de cada símbolo na sequência fornecida. O texto contém os seguintes símbolos e frequências:

Símbolo	Frequência	% aprox.
A	25	53%
B	15	32%
C	4	9%
D	2	4%
E	1	2%
TOTAL	47	100%

Passo 2 — Construção da árvore de Huffman

Ordenamos os símbolos por frequência crescente e unimos os dois menores repetidamente até restar um único nó raiz:

1. Símbolos ordenados: E(1), D(2), C(4), B(15), A(25)
2. Une E(1) + D(2) → nó ED(3)
3. Une ED(3) + C(4) → nó EDC(7)
4. Une EDC(7) + B(15) → nó EDCB(22)
5. Une EDCB(22) + A(25) → raiz(47)

Passo 3 — Atribuição dos códigos

Percorremos a árvore: borda esquerda = 0, borda direita = 1. Os caminhos da raiz até cada folha definem os códigos:

Símbolo	Freq.	Código Huffman	Bits usados
A	25	1	$25 \times 1 = 25$ bits
B	15	01	$15 \times 2 = 30$ bits
C	4	001	$4 \times 3 = 12$ bits
D	2	0001	$2 \times 4 = 8$ bits
E	1	0000	$1 \times 4 = 4$ bits

TOTAL	47	—	79 bits ≈ 10 bytes
-------	----	---	--------------------

Passo 4 — Codificação do texto

Substituindo cada caractere pelo seu código Huffman, o texto AAAABABBBCD... se torna uma sequência binária. O total é **79 bits**, o que exige **10 bytes** (arredondando $79/8 = 9,875 \rightarrow 10$).

Comparação: sem compressão, 47 caracteres $\times$ 8 bits = 376 bits. Com Huffman: 79 bits. Taxa de compressão $\approx 4,75:1$.

Questão 3 (2 pontos)

Elemento estruturante $B = \{(0,0), (0,-1), (0,1)\}$. Calcule a erosão da imagem $I = \{(1,1), (1,3), (2,2), (2,3), (2,4), (3,2), (3,3), (3,4), (4,3)\}$.

Conceito de Erosão

A erosão de I pelo elemento estruturante B é o conjunto de todos os pontos p tais que, ao transladar B para p , todos os elementos de B transladado estejam dentro de I . Formalmente:

$$I \blacksquare B = \{ p \mid B \text{ transladado para } p \subseteq I \}$$

O elemento $B = \{(0,0), (0,-1), (0,1)\}$ é uma barra horizontal de 3 pixels: o pixel central, o da esquerda e o da direita (mesma linha). Logo, um ponto $p = (r, c)$ sobrevive à erosão somente se (r,c) , $(r,c-1)$ e $(r,c+1)$ estiverem todos em I .

Verificação ponto a ponto

Ponto p	Precisa de	Em I?	Fica?
(1,1)	(1,0), (1,1), (1,2)	(1,0) ausente	NÃO
(1,3)	(1,2), (1,3), (1,4)	(1,2) e (1,4) ausentes	NÃO
(2,2)	(2,1), (2,2), (2,3)	(2,1) ausente	NÃO
(2,3)	(2,2), (2,3), (2,4)	TODOS presentes	SIM ✓
(2,4)	(2,3), (2,4), (2,5)	(2,5) ausente	NÃO
(3,2)	(3,1), (3,2), (3,3)	(3,1) ausente	NÃO
(3,3)	(3,2), (3,3), (3,4)	TODOS presentes	SIM ✓
(3,4)	(3,3), (3,4), (3,5)	(3,5) ausente	NÃO
(4,3)	(4,2), (4,3), (4,4)	(4,2) e (4,4) ausentes	NÃO

Resultado: $I \blacksquare B = \{ (2,3), (3,3) \}$

Intuição: a erosão 'encolhe' a forma. Como B é uma barra horizontal, apenas pixels que têm vizinhos imediatos à esquerda e à direita na mesma linha sobrevivem. Isso elimina bordas e pontos isolados.

Questão 4 (2 pontos)

Determine a sequência de operações morfológicas para extrair o contorno interno (que inclui os pontos da forma) de uma imagem.

Conceito de Contorno Interno

Existem três tipos de contorno morfológico: externo, interno e gradiente. O contorno **interno** contém os pixels da borda da forma que ainda pertencem a ela — ou seja, está 'por dentro' da figura.

Sequência de Operações

- 1. Erosão:** aplique a erosão da imagem I pelo elemento estruturante B , obtendo $I \ominus B$. Isso 'encolhe' a forma, eliminando os pixels de borda.
- 2. Subtração:** subtraia o resultado da imagem original: $\text{Contorno_Interno} = I - (I \ominus B)$.

$$\text{Contorno_Interno} = I - (I \ominus B)$$

Por que funciona: a erosão remove exatamente os pixels de borda. Ao subtrair o resultado erodido da imagem original, ficam apenas esses pixels de borda — que são o contorno interno. Todos os pixels que 'sobreviveram' à erosão (interior da forma) são cancelados pela subtração.

Atenção: não confundir com contorno externo $[(I \oplus B) - I]$ nem com gradiente morfológico $[(I \oplus B) - (I \ominus B)]$. A questão pede especificamente o contorno interno.

Resposta: Erosão de I por B , seguida de subtração: $I - (I \ominus B)$.

Questão 5 (2 pontos)

Código de cadeia: 0 = direita, 1 = baixo, 2 = esquerda, 3 = cima. Sequência: 0011110001122222333333 (a) Restitua a imagem. (b) Determine o código normalizado. (c) Calcule o mínimo de bytes.

a) Restituição da imagem

Partindo de um ponto de origem, seguimos cada direção da sequência passo a passo. A sequência tem 22 direções:

Pos.	Código	Direção	Movimento (col, lin)
0→1	0	direita	+1 col
1→2	0	direita	+1 col
2→3	1	baixo	+1 lin
3→4	1	baixo	+1 lin
4→5	1	baixo	+1 lin
5→6	1	baixo	+1 lin
6→7	0	direita	+1 col
7→8	0	direita	+1 col
8→9	0	direita	+1 col

9→10	1	baixo	+1 lin
10→11	1	baixo	+1 lin
11→12	2	esquerda	-1 col
12→13	2	esquerda	-1 col
13→14	2	esquerda	-1 col
14→15	2	esquerda	-1 col
15→16	2	esquerda	-1 col
16→17	3	cima	-1 lin
17→18	3	cima	-1 lin
18→19	3	cima	-1 lin
19→20	3	cima	-1 lin
20→21	3	cima	-1 lin
21→0	3	cima	-1 lin (fecha)

O contorno traça um retângulo fechado de aproximadamente 3×6 pixels: sobe 2 colunas, desce 4 linhas, avança 3 colunas, desce 2 linhas, retorna 5 colunas para esquerda e sobe 6 linhas até o ponto de origem.

b) Código normalizado (menor inteiro)

O código normalizado é a rotação cíclica da sequência que representa o menor número inteiro quando lida da esquerda para a direita. Testamos todas as 22 rotações e escolhemos a que começa com o menor prefixo possível.

Sequência original: 0011110001122222333333

A rotação que começa por mais zeros seguidos produz o menor número. Testando as rotações que iniciam com '0': a que produz o menor inteiro é a que começa com 0001122222333333001111, pois o prefixo 0001... é menor que 0011...

Código normalizado: 0001122222333333001111

c) Mínimo de bytes necessários

A sequência tem **22 símbolos**. Cada símbolo pertence ao conjunto {0, 1, 2, 3}, que possui 4 valores possíveis. Para representar 4 valores são necessários exatamente 2 bits por símbolo (pois $2^2 = 4$).

Total de bits = 22 símbolos × 2 bits/símbolo = 44 bits

Total de bytes = 44 ÷ 8 = 5,5 → arredondando para cima = 6 bytes

Resposta: são necessários no mínimo 6 bytes para codificar o contorno.